

Convolutional Neural Networks (CNN)

Perché le CNN?

Nelle reti neurali classiche (fully connected), per immagini anche di dimensioni normali il numero di connessioni cresce in modo **incontrollabile**, portando a: - **Overfitting** - **Complessità computazionale** eccessiva - **Memoria** insufficiente

Esempio: Esempio Un'immagine 256×256 RGB ha 196,608 pixel. Con un layer FC di 1000 neuroni $\rightarrow$ 196 milioni di pesi!

Le **CNN (Convolutional Neural Networks)** risolvono questo problema sfruttando la **struttura spaziale** delle immagini.

Principi Fondamentali delle CNN

1. Connettività Locale

Invece di connettere ogni neurone a tutti i pixel: - Ogni neurone è connesso solo a una **regione locale** dell'immagine - Questa regione è chiamata **receptive field**

2. Condivisione dei Pesi (Weight Sharing)

- Lo stesso filtro (kernel) è applicato a **tutta l'immagine**
- Riduce drasticamente il numero di parametri
- Invarianza alla traslazione

3. Struttura 3D

Un layer CNN elabora un volume 3D: **height** $\times$ **width** $\times$ **depth** - Il depth corrisponde ai canali (RGB) o alle feature maps

Tipi di Layer nelle CNN

Convolutional Layer (CONV)

Il layer convolutivo applica **filtri (kernel)** all'input.

Parametri: - **Kernel size** (es. 3×3 , 5×5) - **Numero di filtri** (depth dell'output) - **Stride:** passo dello spostamento - **Padding:** bordo aggiunto all'immagine

Formula per la dimensione di output:

$$W_{out} = \frac{W_{in} - F + 2P}{S} + 1$$

Dove: - W = dimensione dell'immagine - F = dimensione del filtro - P = padding - S = stride

Esempio: Esempio Input 32×32 , filtro 5×5 , stride 1, no padding:

$$W_{out} = \frac{32 - 5 + 0}{1} + 1 = 28$$

Per preservare la dimensione: $P = \frac{F-1}{2}$ con $S = 1$

!Pasted image 20251201192025.png

Activation Layer (ReLU)

Applica la funzione ReLU element-wise:

$$f(x) = \max(0, x)$$

- Non modifica le dimensioni
- Introduce la non-linearità necessaria

Pooling Layer (POOL)

Riduce le dimensioni spaziali dell'immagine (sottocampionamento).

Tipi comuni: - **Max Pooling:** prende il valore massimo nella regione - **Average Pooling:** calcola la media

Esempio Max Pooling 2×2 con stride 2: - Riduce le dimensioni a metà - Mantiene le feature più salienti

Info: Scopo del Pooling - Riduce le dimensioni → meno parametri - Aumenta il receptive field dei layer successivi - Introduce invarianza a piccole traslazioni

Fully Connected Layer (FC)

- Identico alle reti neurali classiche
- Usato alla fine della rete per la classificazione
- Collega tutte le feature alle classi di output

Architettura Tipica di una CNN

INPUT → [[CONV → RELU] × N → POOL?] × M → [FC → RELU] × K → FC

Esempio concreto:

INPUT (32×32×3)

↓

CONV (32 filtri 5×5) → 32×32×32

↓

RELU

↓

MAX POOL (2×2) → 16×16×32

↓

CONV (64 filtri 5×5) → 16×16×64

↓

RELU

↓

MAX POOL (2×2) → 8×8×64

↓
 FC → 128
 ↓
 RELU
 ↓
 FC → 10 (classi)
 ↓
 SOFTMAX

Esempio MATLAB

```

layers = [
    imageInputLayer([28 28 1])           % Input: immagine 28×28 grayscale

    convolution2dLayer(3, 8, 'Padding', 'same') % CONV: kernel 3×3, 8 filtri
    reluLayer                               % ReLU
    maxPooling2dLayer(2, 'Stride', 2)       % POOL: 2×2, stride 2

    convolution2dLayer(3, 16, 'Padding', 'same') % CONV: 16 filtri
    reluLayer
    maxPooling2dLayer(2, 'Stride', 2)

    fullyConnectedLayer(2)                 % FC: 2 classi
    softmaxLayer
    classificationLayer()
];

% Opzioni di training
options = trainingOptions('sgdm', ...
    'MaxEpochs', 15, ...
    'InitialLearnRate', 0.0001);

% Addestramento
convnet = trainNetwork(trainDigitData, layers, options);

% Test
YTest = classify(convnet, testImageData);
  
```

Visualizzazione delle Attivazioni

Per capire cosa “vede” la rete, si possono visualizzare le **activation maps**:

```
tf = activations(convnet, trainImageData, 3); % Layer 3
```

!Pasted image 20251201192221.png

!Pasted image 20251201192229.png

!Pasted image 20251201192236.png

Suggerimento: Interpretazione - I primi layer catturano **feature semplici** (edge, texture) - I layer più profondi catturano **feature complesse** (parti di oggetti, forme)

Visualizzazione dei Filtri

I **filtri del primo layer** sono interpretabili visivamente: - Una rete ben addestrata mostra filtri **ben definiti** - Filtri rumorosi indicano **overfitting** o training insufficiente

GPU Computing

Le CNN sfruttano in modo ottimale il **GPU computing**:

Aspetto	CPU	GPU
Core	Pochi (4-16)	Migliaia (1000+)
Paradigma	Sequenziale	Parallelo
Operazioni	Generiche	Matriciali
Memoria	Alta (64+ GB)	Limitata (8-24 GB)

Info: Limite Computazionale Il collo di bottiglia è spesso la **memoria GPU** (8-16 GB). Gli algoritmi di training includono strategie per minimizzare l'uso di memoria (gradient checkpointing, mixed precision).

Architetture Famose

LeNet (1998)

- Prima CNN di successo
- Riconoscimento cifre scritte a mano
- Architettura semplice: CONV → POOL → CONV → POOL → FC

AlexNet (2012)

- Vinse ImageNet 2012 con distacco enorme
- Introdusse ReLU, Dropout, GPU training
- 8 layer, 60 milioni di parametri

VGGNet (2014)

- Kernel piccoli (3×3) impilati
- Reti molto profonde (16-19 layer)
- Dimostrò che la profondità è importante

ResNet (2015)

- Introdusse le **skip connections** (residual learning)
- Permette reti molto profonde (50-152 layer)
- Risolve il problema del vanishing gradient

Best Practice

Citazione: Consiglio da Stanford CS231n *“In practice: use whatever works best on ImageNet. If you’re feeling a bit of a fatigue in thinking about the architectural decisions, you’ll be pleased to know that in 90% or more of applications you should not have to worry about these. Instead of rolling your own architecture for a problem, you should look at whatever architecture currently works best on ImageNet, download a pretrained model and finetune it on your data.”*

Adattamento per Immagini Biomediche

Le CNN sono normalmente progettate per immagini RGB a 8 bit. Per immagini biomediche:

Aspetto	Immagini Standard	Immagini Biomediche
Canali	3 (RGB)	1 (grayscale) o molti (multimodali)
Bit depth	8 bit	12-16 bit
Dimensioni	Variabili	Spesso molto grandi

Adattamenti necessari: - Modificare l’input layer per il numero corretto di canali - **Windowing** per ridurre la dinamica a 8 bit - **Cropping** per focalizzarsi sull’organo di interesse
